

Aufgabe 03 (REST API) - Makerspace-Ausleihe

In dieser Aufgabe geht es um die Umsetzung einer REST API fuer ein kleines Ausleihsystem eines Schul-Makerspaces. Mitglieder koennen Geraete reservieren und ausleihen, Geraete koennen Wartungstickets haben, und die Ausgabe moechte Listen, Filter und einfache Auswertungen ueber die vorhandenen Daten abrufen.

Relevant ist diesmal nur Aufgabe 3. Die Blog-Vorlage wurde durch eine Makerspace-Vorlage ersetzt. Die Models, der `MakerspaceContext`, Seed-Daten sowie DTOs/CMDs sind bereits vorbereitet. Services und Endpoints sollen von Ihnen erstellt werden.

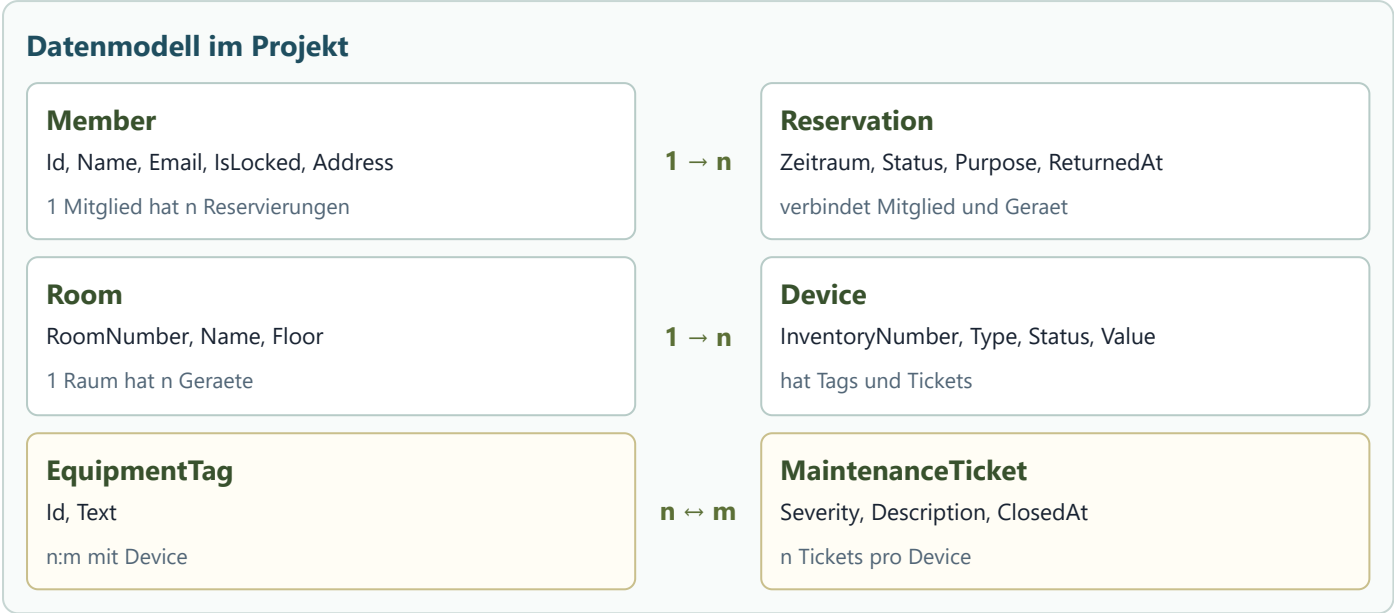
Vorbereitete Dateien

Die folgenden Dateien sind bereits vorhanden und duerfen verwendet bzw. erweitert werden:

Bereich	Datei(en)
Models	<code>src/Aufgabe3_RestfulApi/Model/Device.cs</code> , <code>EquipmentTag.cs</code> , <code>MaintenanceTicket.cs</code> , <code>Member.cs</code> , <code>Reservation.cs</code> , <code>Room.cs</code> , <code>MakerspaceAddress.cs</code> , <code>MakerspaceEnums.cs</code>
DbContext	<code>src/Aufgabe3_RestfulApi/Infrastructure/MakerspaceContext.cs</code>
Seed-Daten	<code>src/Aufgabe3_RestfulApi/Infrastructure/MakerspaceSeeding.cs</code>
DTOs	<code>src/Aufgabe3_RestfulApi/Dtos/MakerspaceDtos.cs</code>
CMDs	<code>src/Aufgabe3_RestfulApi/Cmds/MakerspaceCommands.cs</code>
Mapper optional	<code>src/Aufgabe3_RestfulApi/Mapper/MakerspaceMapper.cs</code>
Services selbst erstellen	<code>src/Aufgabe3_RestfulApi/Services/</code>
Controller optional	<code>src/Aufgabe3_RestfulApi/Controllers/</code>
Tests	<code>tests/Aufgabe3_RestfulApi.Test/TestWebApplicationFactory.cs</code> , <code>TestWebApplicationFactoryDB.cs</code>

Es gibt keine Unterordner wie `Model/Makerspace` oder `Dtos/Makerspace`. Alle Models liegen direkt im Ordner `Model`, alle DTOs direkt im Ordner `Dtos`.

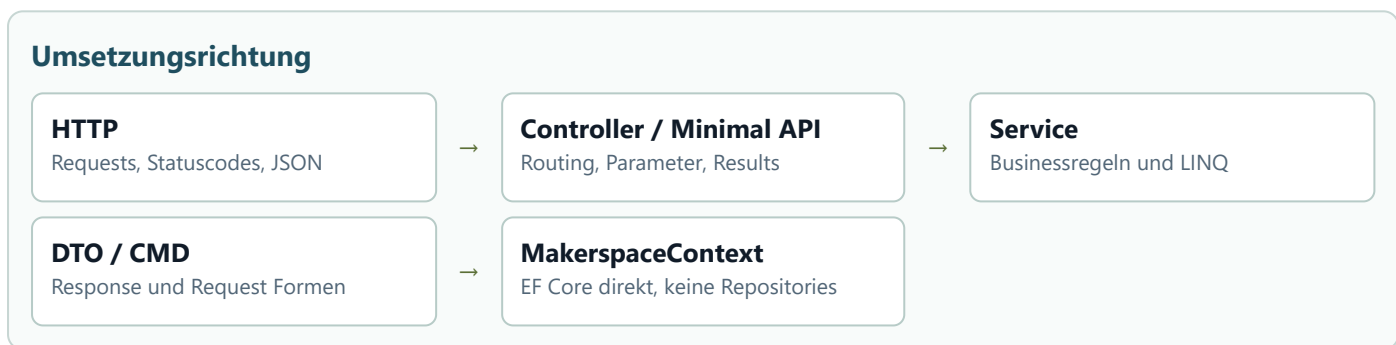
Fachmodell



Attribut	Beschreibung
Id	Eindeutige ID, wird automatisch vergeben
Device	Zugeordnetes Geraet
Member	Zugeordnetes Mitglied
ReservedFrom , ReservedUntil	Zeitraum
ReturnedAt	Rueckgabezeitpunkt, optional
Status	Planned , Active , Completed , Cancelled , Overdue
Purpose	Zweck der Ausleihe
ReturnNote	Rueckgabehinweis, optional

Zusaetzlich gibt es `Room` , `EquipmentTag` und `MaintenanceTicket` . Tags koennen mehreren Geraeten zugeordnet sein, und ein Geraet kann mehrere Tags besitzen.

Vorgaben



Die API muss ueber Services arbeiten. In den Services wird direkt mit dem `MakerspaceContext` gearbeitet; es werden keine Repositories erstellt. LINQ-Abfragen gehoeren in die Services, nicht in Controller oder Minimal-API-Handler.

Mapper duerfen verwendet werden, sind aber nicht verpflichtend. Controller oder Minimal API sind frei wahlbar. Die DTOs und CMDs sind vorgegeben und sollen fuer Requests und Responses verwendet werden.

Es soll mindestens ein Service so getestet werden, dass er gemockt wird. In diesem Fall soll fuer den Test keine Datenbank benoetigt werden. Zusaetzlich sollen Endpunkte auch mit echter Testdatenbank getestet werden.

Zu implementierende Endpunkte

Es sollen sinnngemaess folgende Endpunkte umgesetzt werden:

HTTP	Pfad	Beschreibung
GET	/api/devices	Liefert Geraete als JSON. Filter nach Typ, Status, Suchtext, Mindestwert und Raum sollen moeglich sein. Pagination soll unterstuetzt werden.
GET	/api/devices/{id}	Liefert Details zu einem Geraet inklusive Tags, Raum und offenen Wartungstickets.
POST	/api/devices	Erstellt ein neues Geraet. Inventarnummern sollen eindeutig bleiben.
PUT	/api/devices/{id}/status	Aendert den Status eines Geraets.
DELETE	/api/devices/{id}	Entfernt ein Geraet, sofern es keine aktive Ausleihe dazu gibt.
POST	/api/reservations	Erstellt eine Reservierung fuer ein Geraet und ein Mitglied.
PUT	/api/reservations/{id}/complete	Schliesst eine Ausleihe ab und setzt den Rueckgabezeitpunkt.
GET	/api/members/{id}/reservations	Liefert alle Reservierungen eines Mitglieds; optional nach Status filterbar.
GET	/api/maintenance/open	Liefert offene Wartungstickets; optional nach Schweregrad und Geraetetyp filterbar.
POST	/api/maintenance	Erstellt ein Wartungsticket fuer ein Geraet.
GET	/api/statistics/device-types	Liefert eine einfache Auswertung gruppiert nach Geraetetyp.

Bei einer sehr guten Loesung sind Pagination und HATEOAS-Links bei Listen- und Detailantworten sinnvoll beruecksichtigt.

Beispiele

Beispiel fuer eine Geraeteliste:

```
{
  "items": [
    {
      "id": 1,
      "inventoryNumber": "CAM-2026-001",
      "name": "Sony ZV-E10",
      "deviceType": "Camera",
      "status": "Available",
      "roomNumber": "2.08",
      "replacementValue": 799.90
    }
  ],
  "page": 1,
  "pageSize": 10,
  "totalCount": 1
}
```

Beispiel fuer das Erstellen einer Reservierung:

```
{
  "deviceId": 1,
  "memberId": 2,
  "reservedFrom": "2026-06-12T10:00:00",
  "reservedUntil": "2026-06-12T14:00:00",
  "purpose": "Interview aufnehmen"
}
```

Erwartete Schwerpunkte

Testidee

API-Test mit Mock

Service ersetzen, keine DB notwendig

Service-Test

LINQ, Regeln, Fehlersituationen

API-Test mit Testdatenbank

SQLite, Seeding, JSON und Statuscodes pruefen

Die Loesung soll zeigen, dass folgende Dinge beherrscht werden:

- Services mit direkter Verwendung des `DbContext`
- LINQ in Services: Filtern, Suchen, Sortieren, Gruppieren, Pagination
- Verwendung von DTOs und CMDs
- Statuscodes passend zur Situation (`200` , `201` , `204` , `400` , `404` , `409`)
- Validierung einfacher Regeln, z.B. Zeitraum, gesperrtes Mitglied, nicht verfuegbares Geraet
- REST-Endpunkte ueber Controller oder Minimal API
- Tests mit gemocktem Service und Tests mit echter Testdatenbank

Auch diese Implementierung ist zu testen.